

Recomendación de Infraestructura

Base de datos y hosting para el Sistema de Control de Asistencia NOM-037 con operación de 20 empleados.

RESUMEN EJECUTIVO

La infraestructura actual — **Vercel + Supabase PostgreSQL + Next.js 16** — es técnicamente óptima para la operación proyectada. **No se recomienda migrar.** El sistema usa menos del 0.1% de los límites del plan gratuito, cumple con NOM-037 y LFT, y su costo de operación es prácticamente nulo.

FECHA	VERSIÓN	ALCANCE
Julio 2026	1.0	20 empleados

Resumen Ejecutivo

Después de analizar la carga proyectada de operación — 20 empleados, registro diario de asistencia, vacaciones, auditoría y reportes mensuales —, la conclusión es contundente: **la infraestructura actual (Supabase PostgreSQL + Vercel) es técnicamente la opción óptima**. No se recomienda migrar a otra base de datos ni cambiar de hosting en este momento.

El sistema actual está usando menos del 0.1% de la capacidad disponible, cumple con los requisitos legales de NOM-037 y LFT en cuanto a trazabilidad e integridad de datos, y su costo de operación es prácticamente nulo para el volumen del cliente. Una migración representaría semanas de trabajo, riesgo de pérdida de datos y degradación de características críticas, sin ningún beneficio tangible.

Este documento detalla el análisis técnico detrás de esta recomendación, explica por qué Vercel es la plataforma adecuada para alojar la aplicación, y analiza qué sucedería en caso de migrar a otro proveedor de hosting.

Recomendación sobre Base de Datos

2.1 Configuración actual

El sistema está desplegado sobre **Supabase**, que es **PostgreSQL 15** gestionado en la nube de **AWS**. La conexión se realiza a través de PgBouncer (transaction pooler en el puerto 6543), que es el estándar recomendado para aplicaciones serverless como las que corren en Vercel.

La base de datos contiene 10 tablas principales: `Company`, `User`, `Sucursal`, `Employee`, `WorkSchedule`, `AttendanceRecord`, `Vacation`, `Holiday`, `AuditLog` y `DynamicQR`. Todas están correctamente indexadas y con constraints de integridad (llaves foráneas, campos únicos como `@unique([employeeId, date])` en registros de asistencia).

2.2 Análisis de carga para 20 empleados

Para dimensionar correctamente, se calculó la carga operativa real del sistema con 20 empleados activos en operación diaria:

Concepto	Cálculo	Resultado
Registros de asistencia por día	20 empleados × ~4 eventos (entrada, comida, descanso, salida)	~80 inserciones/día
QRs dinámicos generados por día	1 por sucursal, rotación cada 60s × 10 horas laborables	600 a 1,200/día (se purgan)
Logs de auditoría por día	Logins + check-ins + cambios de configuración	50 a 100/día
Solicitudes de vacaciones	Baja frecuencia operativa	5 a 10/mes
Registros de asistencia por año	20 empleados × ~260 días hábiles	~5,200 registros/año
Tamaño total estimado de BD por año	Attendance + audit + QR purgado	Menos de 50 MB/año

2.3 Veredicto

Supabase soporta esta carga sin ningún problema técnico.

Para ponerlo en perspectiva, PostgreSQL en hardware modesto maneja **miles de inserciones por segundo**. El pico máximo del sistema (20 check-ins casi simultáneos a las 8:00 AM) se resuelve en milisegundos. El plan gratuito de Supabase (500 MB de almacenamiento, 2 GB de egresos mensuales) cubre con holgura **varios años de operación** antes de acercarse a cualquier límite.

2.4 Por qué PostgreSQL es la elección correcta

Las características técnicas de PostgreSQL se alinean de forma natural con los requisitos de un sistema de control de asistencia sujeto a regulación mexicana (NOM-037 y LFT). Cada característica del motor resuelve un requisito concreto del dominio:

- **Transacciones ACID.** Garantizan que un registro de check-in no quede a medio escribir. Si se registra la entrada pero falla la salida, la base de datos revierte la operación parcial. Esto es crítico para nómina, donde un registro inconsistente significa pago incorrecto.
- **Constraints únicos a nivel base de datos.** El constraint `@@unique([employeeId, date])` impide que un empleado tenga dos registros el mismo día, incluso si hay un bug en la aplicación. La integridad de datos no depende del código, sino de la base de datos.
- **Soporte JSONB.** El campo `details` de `AuditLog` puede crecer con metadatos sin requerir migraciones de schema cada vez que se agrega un evento nuevo.
- **Backups automáticos y Point-in-Time Recovery.** Supabase realiza backup diario y permite recuperación a cualquier momento dentro de los últimos 7 días (Plan Pro). La NOM-037 exige conservar registros durante años; esta característica los protege contra fallas técnicas.
- **Row Level Security (RLS).** Permite definir políticas a nivel base de datos para que un administrador de sucursal jamás vea datos de otra sucursal, incluso si existe un bug en el código de la aplicación. Es defensa en profundidad, no solo seguridad en capa aplicativa.
- **Pooler de conexiones (PgBouncer).** Ya configurado, reutiliza conexiones TCP, lo que es esencial para Vercel serverless donde cada función puede abrir su propia conexión.

2.5 Comparativa con otras bases de datos

Para confirmar que la recomendación es la correcta, se evaluaron las alternativas más comunes en el mercado:

Base de datos	¿Sirve para 20 empleados?	Veredicto
PostgreSQL en Supabase (actual)	Excelente	Mantener. Es la opción óptima.
MySQL (PlanetScale, AWS RDS)	Sí	Migrar no aporta nada y se pierden constraints avanzadas y RLS.
MongoDB (NoSQL)	Funciona, pero con riesgo	Pierde transacciones ACID y constraints únicos. Desaconsejado para nómina y datos legales.
SQLite (Turso)	Sí para desarrollo	Insuficiente para producción multiusuario concurrente.
CockroachDB	Sí, pero excesivo	Distribución global que no se necesita y encarece la operación.
SQL Server / Oracle	Sí	Costoso y propietario, sin beneficio real sobre PostgreSQL.

2.6 ¿Cuándo sí habría que considerar migrar?

La migración solo se justificaría en escenarios que **no aplican** al cliente actual. Es importante conocerlos para dimensionar correctamente la decisión:

- **Más de 500 empleados** con check-in masivo simultáneo (ej. turnos de fábrica). Aún así, PostgreSQL seguiría siendo correcto, pero se añadirían réplicas de lectura y caché Redis.
- **Carga sostenida mayor a 1,000 inserciones por segundo.** Aquí se implementaría particionamiento por fecha en `AttendanceRecord` y `AuditLog`, pero manteniendo PostgreSQL.
- **Operación multi-empresa (SaaS)** con miles de organizaciones. Postgres con schema-per-tenant o particionamiento. Sigue siendo Postgres.
- **Latencia geográfica** porque los empleados están en otro continente. Se movería el proyecto Supabase a la región más cercana, sin cambiar de motor.
- **Necesidad de análisis OLAP pesado** (dashboards en tiempo real con agregaciones complejas sobre millones de registros). Se añadiría un data warehouse como réplica de lectura, no se reemplazaría Postgres.

El cliente está a aproximadamente **dos órdenes de magnitud del primer umbral** (500 empleados). La decisión de migrar no se justifica técnicamente.

2.7 Recomendaciones de optimización (sin migrar)

Aunque no se requiere migrar, sí se recomiendan tres acciones de mantenimiento preventivo para blindar la operación a largo plazo:

Acciones recomendadas

1. **Subir al plan Pro de Supabase (\$25 USD/mes)** para tener backups automáticos diarios y Point-in-Time Recovery. Esto protege contra errores humanos y fallas técnicas.
2. **Configurar un cron mensual** que purgue los `DynamicQR` vencidos con más de 30 días y los `AuditLog` con más de 2 años. Evita crecimiento innecesario de la base de datos.
3. **Mantener el backup manual mensual** exportado a JSON (como el ya existente en `backups/`) como capa adicional de respaldo fuera de Supabase.

Recomendación sobre Hosting (Vercel)

3.1 Configuración actual

La aplicación está desplegada en **Vercel**, región `iad1` (este de EE. UU., la región más cercana a México con presencia de Vercel). El build ejecuta `prisma generate && prisma db push && next build`, lo que asegura que el cliente Prisma se regenere en cada deploy. El framework detectado es Next.js 16, con soporte nativo para Server Components, API routes y middleware.

3.2 Por qué Vercel es la plataforma adecuada

Vercel no es solo un hosting: es la plataforma diseñada por los mismos creadores de Next.js. Esto se traduce en ventajas concretas que otras plataformas no replican al mismo nivel:

- **Compatibilidad nativa con Next.js 16.** Server Components, App Router, middleware, ISR (Incremental Static Regeneration), edge functions y streaming SSR funcionan out-of-the-box sin configuración. Otra plataforma requiere adaptadores que pueden romperse en cada actualización de Next.js.
- **Despliegue automático desde Git.** Cada push a `main` se despliega a producción. Cada pull request genera un preview deployment con URL única. Esto significa cero intervención manual para publicar cambios.
- **Edge Network global.** La aplicación se sirve desde el CDN más cercano al usuario. Para empleados en México, el tiempo de carga inicial es menor a 200 ms.
- **Serverless Functions escalado automático.** Si un día todos los empleados hacen check-in al mismo tiempo, Vercel escala automáticamente las funciones sin intervención. No hay que dimensionar servidores.
- **Variables de entorno cifradas y por entorno.** Separación nativa entre producción, preview y desarrollo.
- **Monitoreo y logs integrados.** Cada función serverless tiene logs en tiempo real, métricas de duración y errores. No requiere configurar Datadog o Sentry para empezar.
- **Plan gratuito suficiente.** Para 20 empleados, el plan Hobby (gratuito) cubre todo: 100 GB de ancho de banda, 100 GB-horas de serverless, dominios personalizados.

3.3 Qué pasa si se migra a otro hosting

La pregunta clave es: **¿qué se pierde al irse de Vercel?** A continuación, el análisis honesto de cada alternativa común:

Opción A: Self-hosted en VPS (DigitalOcean, Linode, Hetzner)

Qué se gana: Control total del servidor. Costo fijo predecible (VPS desde \$5 USD/mes).

Qué se pierde: Hay que configurar manualmente Node.js, Nginx reverse proxy, certificados SSL (LetsEncrypt), PM2 para mantener el proceso vivo, logrotate, firewall y backups del servidor. Se pierde el escalado automático: si el VPS tiene 1 GB de RAM y entran 50 empleados al mismo tiempo, el servidor puede quedar sin responder. Hay que mantener el sistema operativo actualizado (security patches, kernel upgrades). No hay preview deployments; cada cambio se publica directo a producción o hay que montar un CI/CD propio.

Veredicto: Solo vale la pena si se tiene personal técnico dedicado a sysadmin. Para una pyme sin DevOps, es un retroceso.

Opción B: Railway / Render

Qué se gana: Deploy más simple que un VPS. Soporte nativo para Node.js.

Qué se pierde: No tienen el nivel de integración con Next.js que tiene Vercel. Funciona, pero hay que adaptar la configuración. Preview deployments existen pero son menos potentes. El plan gratuito de Render expira tras 15 minutos de inactividad (cold starts de 30+ segundos en el primer request del día). Planes pagados inician en 7 — 19 USD/mes por servicio, más base de datos, más storage. Para igualar features de Vercel gratuito, se paga 30-50 USD/mes.

Veredicto: Alternativa razonable si Vercel deja de ser gratuito, pero inferior para Next.js.

Opción C: Cloudflare Pages / Workers

Qué se gana: Performance excepcional (red edge global). Plan gratuito muy generoso.

Qué se pierde: Next.js 16 aún no es 100% compatible con Cloudflare Pages. Hay limitaciones con Server Components, middleware y algunas APIs. Requiere adaptador `@cloudflare/next-on-pages` que va detrás de cada release de Next. Las serverless functions de Cloudflare corren en V8 isolates, no en Node.js completo. Algunas librerías de Node (como `bcrypt`, `crypto` nativo) no funcionan sin polyfills. WebSocket (socket.io del mini-service realtime) requiere configuración adicional con Durable Objects.

Veredicto: Prometedor a futuro, pero inmaduro para Next.js 16 hoy. No recomendado para producción estable.

Opción D: Netlify

Qué se gana: Similar a Vercel en UX del dashboard.

Qué se pierde: El soporte de Next.js en Netlify es a través del plugin `@netlify/plugin-nextjs`, que va un paso detrás de Vercel. Server Components y App Router tienen bugs documentados. Serverless functions tienen timeout de 10 segundos en plan gratuito (Vercel: 60 segundos en Hobby, 300 en Pro). Para reportes mensuales que generan Excel/PDF, esto puede ser insuficiente. Edge functions menos maduras que Vercel.

Veredicto: Peor que Vercel para Next.js. Solo se justifica si ya se tiene todo el stack en Netlify.

Opción E: AWS (Amplify, EC2, ECS)

Qué se gana: Potencia técnica ilimitada. Integración con otros servicios AWS (RDS, S3, Lambda).

Qué se pierde: Complejidad operativa alta. Configurar Amplify para Next.js 16 es posible pero tedioso. Facturación impredecible (pago por uso sin techo claro). Requiere conocimientos de AWS IAM, VPC, security groups. El soporte de Next.js App Router en Amplify tiene limitaciones conocidas.

Veredicto: Overkill para 20 empleados. Solo se justifica si la empresa ya tiene infraestructura AWS y personal DevOps.

3.4 Tabla resumen: Vercel vs alternativas

Plataforma	Soporte Next.js 16	Costo mensual	Esfuerzo DevOps	Veredicto
Vercel (actual)	Nativo, día cero	\$0 (Hobby)	Cero	Mantener
VPS (DigitalOcean)	Manual, con Nginx	\$5-20	Alto	No recomendado
Railway	Bueno	\$5-20	Bajo	Alternativa aceptable
Render	Bueno	\$7-19 por servicio	Bajo	Alternativa aceptable
Cloudflare Pages	Parcial	\$0	Medio	Inmaduro
Netlify	A través de plugin	\$0-19	Bajo	Inferior a Vercel
AWS Amplify	Limitado	\$5-30	Alto	Overkill

3.5 Costo real de migrar de Vercel

Si el cliente decidiera migrar a otra plataforma, el costo real sería significativo en múltiples dimensiones:

- **Tiempo de ingeniería:** 1 a 2 semanas para configurar el nuevo hosting, adaptar el build, configurar dominio, SSL, variables de entorno y probar end-to-end.
- **Riesgo de downtime:** Migración DNS implica propagación de 24-48 horas donde la app puede estar parcialmente disponible.
- **Riesgo de pérdida de datos:** Si la migración involucra mover también la base de datos, hay que reconfigurar todas las URLs de conexión y hay ventana de riesgo.
- **Pérdida de features:** Sin preview deployments, sin escalado automático, sin logs integrados.
- **Costo financiero neto:** Negativo. Vercel Hobby es gratuito; cualquier alternativa pagada o self-hosted con tiempo de ingeniería resulta más cara.

3.6 Recomendación final sobre hosting

Quedarse en Vercel.

Las únicas razones válidas para migrar serían: (1) Vercel elimina el plan Hobby gratuito (no hay indicios de esto); (2) la empresa crece a más de 1,000 empleados y necesita features enterprise (en cuyo caso Vercel Pro a \$20 USD/mes sigue siendo mejor que migrar); o (3) razones de soberanía de datos que exijan hospedar en México (en cuyo caso, la respuesta sería un VPS en AWS México o un proveedor local, aceptando el costo operativo). Ninguna aplica al cliente actual.

Conclusión General

La infraestructura actual del sistema de control de asistencia — **Vercel (hosting) + Supabase PostgreSQL (base de datos) + Prisma (ORM) + Next.js 16 (framework)** — es técnicamente la opción óptima para una operación de 20 empleados. **No se recomienda migrar a ninguna otra base de datos ni cambiar de hosting.**

Las razones son claras y cuantificables:

1. **Capacidad sobrada:** El sistema usa menos del 0.1% de los límites del plan gratuito.
2. **Cumplimiento legal:** PostgreSQL garantiza integridad ACID y trazabilidad requerida por NOM-037 y LFT.
3. **Costo cero:** Tanto Vercel Hobby como Supabase Free son gratuitos para este volumen.
4. **Cero mantenimiento operativo:** No hay servidores que administrar, ni SSL que renovar, ni backups que configurar manualmente.
5. **Escalabilidad futura:** Hay margen para crecer hasta 500+ empleados sin tocar la arquitectura.

Lo que sí se recomienda hacer como mantenimiento preventivo (sin migrar):

- Subir a Supabase Pro (\$25 USD/mes) para backups automáticos diarios.
- Configurar un cron mensual de purga de QRs vencidos y logs antiguos.
- Mantener el backup manual mensual en JSON como capa adicional.

Decisión final

Con estas tres acciones, la infraestructura queda **blindada para operación continua durante años** sin necesidad de migraciones. La recomendación técnica formal y definitiva es: **mantener Vercel + Supabase PostgreSQL.**